

Guía de React Router — CloudWiki

Esta guía recorre los cambios reales de CloudWiki y te enseña a repetirlos. La aplicación ahora tiene la portada, una página de conceptos y una respuesta para direcciones desconocidas. Las rutas `/servicios` y `/proveedores` **no** están implementadas: aparecerán únicamente como ejercicios.

1. ¿Qué problema resuelve React Router?

Antes de esta etapa, CloudWiki mostraba una sola página: la landing page. Sus enlaces del menú, como `#services` y `#providers`, movían al usuario a distintas secciones de ese mismo documento. Eso funciona bien para una página larga, pero no crea páginas independientes con sus propias direcciones.

En una aplicación con múltiples páginas, una URL puede representar cada vista. Por ejemplo, `/` representa el inicio y `/conceptos` representa una página dedicada a conceptos. Hace falta una herramienta que lea la dirección y elija qué contenido de React mostrar.

CloudWiki es una **SPA** (aplicación de una sola página): el navegador carga el documento inicial y React actualiza la vista dentro de ese documento. Al navegar con React Router, normalmente cambia la URL y el componente visible sin solicitar y reconstruir un documento HTML entero para cada enlace. Esto suele hacer que la navegación se sienta más fluida. En cambio, un enlace HTML convencional puede pedir otra página al servidor y hacer que el navegador vuelva a cargar el documento.

La palabra «página» en una SPA suele referirse a una vista asociada a una URL y representada por un componente React, no necesariamente a un archivo HTML distinto.

2. ¿Qué es React Router?

React Router es una biblioteca que conecta las direcciones del navegador con los componentes de React. El router observa la URL actual; las rutas declaradas en la aplicación describen qué componente corresponde a cada dirección.

Puedes imaginarlo como un recepcionista: recibe la dirección solicitada, busca la regla que coincide y dirige al visitante al componente adecuado. La analogía termina ahí: en la aplicación, son los componentes de React los que producen la interfaz.

- **¿Qué es?** Una biblioteca de navegación y enrutamiento para React.
- **¿Para qué sirve?** Para representar distintas vistas según la URL y navegar entre ellas.
- **¿Dónde lo usamos?** `BrowserRouter` se monta en `src/main.jsx`; las reglas están en `src/App.jsx`.
- **¿Cómo se reutiliza?** Se instala en una aplicación React, se coloca un router alrededor de la interfaz y se declaran sus rutas.

3. Antes y después de React Router

Antes, el punto de entrada montaba directamente `App`, que contenía toda la landing:

```
Usuario
↓
```

```
React
  ↓
App.jsx
  ↓
Landing Page
```

Los enlaces del menú apuntaban a secciones de la misma landing, por ejemplo `#services`.

Ahora, `BrowserRouter` proporciona el contexto de navegación, y las rutas determinan cuál componente de página mostrar:

```
Usuario
  ↓
BrowserRouter
  ↓
URL solicitada
  ↓
Routes y Route
  ↓
Home, Conceptos o página 404
```

La landing original se conservó y ahora vive en `src/pages/Home.jsx`. La barra de navegación y el pie se comparten alrededor de las rutas.

4. Instalación

Se agregó `react-router`, la biblioteca necesaria para usar el enrutamiento de React en este proyecto. La versión guardada en `package.json` es `^7.18.4`, compatible con la versión de React de CloudWiki y con Node.js 22.16.0, el entorno de este proyecto. Aunque npm ofrece una versión principal más nueva, esta requiere Node.js 22.22.0 o posterior; por eso se eligió la versión 7, que requiere Node.js 20 o posterior y funciona con el entorno disponible. El archivo `package-lock.json` registra la resolución exacta de la instalación.

Para instalarla en un proyecto similar desde su carpeta raíz, ejecuta:

```
npm install react-router@7.18.4
```

En CloudWiki el cambio de dependencia en `package.json` queda así:

```
"dependencies": {
  "react": "^19.2.8",
  "react-dom": "^19.2.8",
  "react-router": "^7.18.4"
}
```

No se agregó ninguna otra dependencia.

5. BrowserRouter

BrowserRouter conecta React Router con la barra de direcciones del navegador. Usa el historial del navegador para que la URL pueda cambiar durante la navegación y permite que los componentes descendientes utilicen las herramientas del router.

Se encuentra en `src/main.jsx`, cerca del punto de entrada. Así toda la aplicación —incluidos **App**, la navegación y las páginas— puede usar React Router:

```
import { BrowserRouter } from 'react-router'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

Sin este proveedor, los componentes de rutas y los enlaces no tendrían el contexto del router que necesitan. Por eso normalmente se coloca una sola vez, por encima de las rutas, en el punto de entrada.

6. Routes

Routes agrupa las reglas de navegación. Cuando cambia la URL, busca entre sus **Route** la que corresponda y representa su elemento.

En CloudWiki, `src/App.jsx` conserva la barra y el pie comunes y coloca **Routes** en el área principal:

```
<main>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/conceptos" element={<Conceptos />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
</main>
```

Piensa en **Routes** como el lugar donde el router consulta las reglas disponibles. No es una página ni contiene por sí mismo el contenido que ve el usuario.

7. Route

Route declara una regla que asocia un patrón de URL con un elemento de React.

```
<Route path="/conceptos" element={<Conceptos />} />
```

- **path**: el camino de la URL que debe coincidir; aquí es `/conceptos`.
- **element**: el elemento de React que se representa cuando coincide; aquí es `<Conceptos />`.
- **Componente**: una función de React que devuelve interfaz. En este ejemplo, `Conceptos` es el componente y `<Conceptos />` es el elemento que se pasa a la ruta.
- **URL**: la dirección que el navegador está visitando. Si la ruta coincide con `/conceptos`, se representa el elemento configurado para ella.

Una URL y un componente están relacionados por la regla, pero no son lo mismo: la URL identifica la dirección; el componente produce el contenido.

8. La página Conceptos

La página está en `src/pages/Conceptos.jsx`. Se separó de la landing para que tenga un componente de página propio y para que el ejemplo de ruta sea fácil de reconocer. Devuelve JSX con un título, una introducción y cuatro tarjetas: Virtualización, Contenedores, Serverless y Cloud Storage. Reutiliza clases de tarjetas y encabezados ya presentes en CloudWiki.

El recorrido es:

```
URL /conceptos
  ↓
BrowserRouter
  ↓
Route con path="/conceptos"
  ↓
Componente Conceptos
  ↓
JSX de la página
  ↓
Interfaz mostrada por el navegador
```

La regla que conecta la URL con el componente está en `src/App.jsx`; el contenido JSX está en `src/pages/Conceptos.jsx`.

9. Link

`Link` es el componente de React Router para navegar a otra ruta interna. Se comporta visualmente como un enlace, pero deja que React Router actualice la URL y la vista dentro de la SPA.

En la navegación de CloudWiki se usa así:

```
<Link to="/conceptos">Conceptos</Link>
```

La propiedad `to` indica el destino. Para navegar a la portada se usa `to="/"`.

La alternativa HTML tradicional sería:

```
<a href="/conceptos">Conceptos</a>
```

Para las rutas internas de esta SPA se prefiere `Link`: permite que React Router gestione el cambio de vista sin una recarga completa del documento. Los enlaces a secciones existentes de la landing, como `#services`, se conservaron como anchors porque no son rutas nuevas: apuntan a elementos de esa misma página.

10. NavLink

CloudWiki **no** usa `NavLink`; utiliza `Link` para Inicio y Conceptos. `NavLink` es otra opción de React Router para enlaces de navegación. Además de navegar, permite detectar si el destino está activo; es útil cuando se quiere resaltar en el menú la página que se está visitando, por ejemplo con un estilo o una indicación accesible. No hace falta cambiar a `NavLink` si ese estado visual no se necesita.

11. Ruta 404

La última regla de `src/App.jsx` es:

```
<Route path="*" element={ <NotFound /> } />
```

El patrón `*` actúa como comodín: coincide cuando ninguna de las rutas anteriores corresponde a la dirección. En ese caso se representa `src/pages/NotFound.jsx`, que muestra «Página no encontrada» y un enlace para volver al inicio.

Una ruta 404 mejora la experiencia porque una dirección mal escrita no deja al usuario ante una pantalla vacía. No se debe colocar antes de las rutas concretas: como comodín, debe ser la alternativa final.

12. Estructura de carpetas

Esta es la estructura real relevante para las páginas, los componentes y los estilos tras esta etapa:

```
src/  
├── assets/  
├── components/  
│   ├── CallToAction.jsx  
│   ├── CloudProviders.jsx  
│   ├── DeploymentModels.jsx  
│   ├── Footer.jsx  
│   ├── Hero.jsx  
│   ├── Introduction.jsx  
│   ├── Navbar.jsx  
│   ├── ServiceModels.jsx  
│   └── WikiTopics.jsx  
└── data/
```

```
|   └─ siteContent.js
└─ pages/
    ├── Conceptos.jsx
    ├── Home.jsx
    └─ NotFound.jsx
└─ App.css
└─ App.jsx
└─ index.css
└─ main.jsx
```

`components/` guarda piezas reutilizables; `pages/` contiene las vistas asociadas a rutas; `data/` contiene contenido compartido. `main.jsx` inicia React y el router, mientras `App.jsx` compone la navegación compartida y las reglas.

13. ¿Cómo creo yo una nueva ruta?

Practica con `/servicios`. **No existe todavía en el código**; los fragmentos siguientes son instrucciones para que la crees tú.

Paso 1: crear el componente de página

Crea `src/pages/Servicios.jsx`. Empieza con un componente React que devuelva una sección con un encabezado y una introducción:

```
function Servicios() {
  return (
    <section className="content-section">
      <div className="container">
        <div className="section-heading centered page-heading">
          <h1>Modelos de servicio en la nube</h1>
          <p>Escribe aquí una introducción breve.</p>
        </div>
      </div>
    </section>
  )
}

export default Servicios
```

Este es un punto de partida, no una solución completa para el ejercicio. Más adelante puedes agregar tarjetas reutilizando las clases de CloudWiki.

Paso 2: importar el componente y registrar la ruta

En `src/App.jsx`, importa `Servicios` desde `./pages/Servicios` y agrega una regla concreta dentro de `Routes`:

```
<Route path="/servicios" element={<Servicios />} />
```

Colócala junto a Inicio y Conceptos, antes de la ruta comodín `path="*"`. La ruta relaciona la dirección y el componente; no crea el archivo por ti.

Paso 3: agregar el enlace

En `src/data/siteContent.js` ya existe un elemento «Servicios» que hoy apunta al anchor `#services`. Cuando la nueva página esté lista, cambia el destino de ese elemento existente a `/servicios`:

```
{ label: 'Servicios', href: '/servicios' }
```

No agregues una segunda entrada «Servicios»: reemplaza la existente para evitar enlaces duplicados. La navegación de CloudWiki ya representa con `Link` los destinos que empiezan con `/`; por eso esta entrada interna se mostrará como un enlace SPA. Ten en cuenta que, al cambiar el destino, ese elemento dejará de saltar a la sección `#services` de Inicio.

Paso 4: completar el contenido

Vuelve a `Servicios.jsx` y desarrolla el contenido que pide el ejercicio: título, introducción y tarjetas de IaaS, PaaS y SaaS. Puedes usar la estructura visual de las tarjetas existentes, sin crear páginas independientes para cada modelo.

Paso 5: probar la URL

Inicia el servidor con `npm run dev`, abre la dirección local que Vite muestre y visita `/servicios`. Comprueba también que el nuevo enlace llega a la misma página, que Inicio sigue funcionando y que la página 404 continúa apareciendo para una URL desconocida.

14. ¿Cómo crear una tercera ruta?

El proceso se repite para `/proveedores`: crea `src/pages/Proveedores.jsx`, impórtalo en `src/App.jsx`, registra `<Route path="/proveedores" element={<Proveedores />} />` antes del comodín y cambia el destino del elemento «Proveedores» ya existente en `navItems` de `#providers` a `/proveedores`. No dupliques el elemento.

Después escribe el contenido que quieras para AWS, Microsoft Azure y Google Cloud, y prueba la ruta y el enlace. Este procedimiento no requiere cambiar `BrowserRouter`; ya envuelve toda la aplicación.

15. Errores comunes

- **Olvidar `BrowserRouter`**: el código de rutas o enlaces no tiene el proveedor necesario. Comprueba que `App` está dentro de `BrowserRouter` en `src/main.jsx`.
- **Escribir mal `path`**: `/concepto` no coincide con `/conceptos`. Compara la ruta registrada, el destino del enlace y la URL carácter por carácter.

- **Usar `<a href="/...">` para navegar a otra página interna:** puede recargar el documento. Usa `Link` para las rutas SPA. Los anchors existentes para `#services` y otras secciones siguen siendo anchors.
- **Crear el componente y olvidar registrar la ruta:** el archivo existe, pero el router no sabe qué URL debe mostrarlo.
- **Registrar la ruta y olvidar el componente o su importación:** React no podrá representar el elemento indicado.
- **Confundir la ruta con el componente:** `path` describe una URL; `element` indica qué interfaz se muestra cuando coincide.
- **Escribir una URL que no existe:** debe aparecer la página comodín 404. Si esperabas una página concreta, revisa el registro de su ruta.
- **Recibir un 404 del servidor al desplegar en Vercel:** puede ocurrir si el servidor busca un archivo físico para `/conceptos` en lugar de servir la SPA. La sección siguiente explica el rewrite incluido en este proyecto.

16. React Router + Vercel

Al navegar desde `/` con `Link`, React Router puede mostrar `/conceptos` sin pedir una página HTML nueva. Pero si alguien escribe `/conceptos` en la barra, abre un marcador o actualiza esa página, la primera petición llega al servidor con esa ruta.

Si el servidor intenta buscar un archivo físico llamado `/conceptos`, no lo encuentra. Para que React Router pueda decidir qué mostrar, Vercel debe entregar primero el documento principal de la SPA (`index.html`); después se ejecuta React y el router reconoce la URL `/conceptos`.

CloudWiki incluye `vercel.json` en la raíz con la regla de rewrite recomendada por Vercel para una SPA Vite:

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

Un rewrite sirve el documento de la aplicación sin cambiar la URL que ve el usuario. Por eso se conserva `/conceptos` y el router cliente puede mostrar la página correcta. Sin esta regla, el acceso directo o la recarga de una ruta profunda puede dar un 404 del servidor, aunque la navegación desde la portada funcione. Tras desplegar, prueba ambos casos en la URL de producción.

17. Ejercicio para mí

Implementa `/servicios` sin copiar una solución completa:

- Crea una página con un título y una introducción.
- Agrega tres tarjetas: IaaS, PaaS y SaaS.

- Cambia el destino del enlace «Servicios» que ya existe en la navegación para que abra `/servicios` (no agregues un duplicado).
- Registra la ruta para que `/servicios` muestre esa página.
- Comprueba la navegación desde el menú y al escribir directamente la URL.
- Verifica que Inicio, Conceptos y la ruta 404 siguen funcionando.

18. Segundo ejercicio

Implementa `/proveedores`:

- Crea un componente de página separado.
- Muestra AWS, Microsoft Azure y Google Cloud.
- Registra la ruta y cambia el destino del enlace «Proveedores» ya existente para que abra `/proveedores` (no agregues un duplicado).
- Comprueba el acceso desde el menú y mediante la URL directa.

No hay una solución incluida: utiliza el proceso del ejercicio anterior como guía.

19. Tabla de consulta rápida

Concepto	Para qué sirve	Ejemplo
<code>BrowserRouter</code>	Conecta React Router con la URL y el historial del navegador.	<code><BrowserRouter><App /></BrowserRouter></code>
<code>Routes</code>	Agrupar las reglas y muestra la ruta que coincide.	<code><Routes>...</Routes></code>
<code>Route</code>	Relaciona un patrón de URL con un elemento React.	<code><Route path="/conceptos" element={<Conceptos />} /></code>
<code>Link</code>	Navega a una ruta interna sin recargar la SPA.	<code><Link to="/conceptos">Conceptos</Link></code>
<code>NavLink</code>	Navega como <code>Link</code> y permite reconocer cuándo su destino está activo.	Útil para resaltar la página actual en un menú.
<code>path</code>	Declara qué URL debe coincidir con una ruta.	<code>path="/conceptos"</code>
<code>element</code>	Indica qué elemento React se muestra al coincidir.	<code>element={<Conceptos />}</code>
<code>404</code>	Presenta una alternativa cuando ninguna ruta coincide.	<code><Route path="*" element={<NotFound />} /></code>
<code>SPA</code>	Aplicación que actualiza vistas en el cliente dentro del documento principal.	CloudWiki carga <code>index.html</code> y React Router muestra sus páginas.